



Aviation Accident Analysis Using Large Language Models

Kanu Shetkar & Sankaran Mahadevan

Agenda

- NTSB background & data
- Prior paper (Zhang and Mahadevan 2021)
- Bayesian Networks: simple example
- Previous approach and Limitations
- Current approach
- Model structure overview
 - Diagnosis mode
 - Prognosis mode
- Demo
- Evaluation, Limitations, Future Work



NTSB Overview

National Transportation Safety Board:

- Independent U.S. agency (est. 1967)
- Investigates transportation accidents



Prior Paper



- Bayesian Network on NTSB coded data
- Backward inference: infer likely causes from outcomes
- Forward inference: conditional probabilities of downstream events

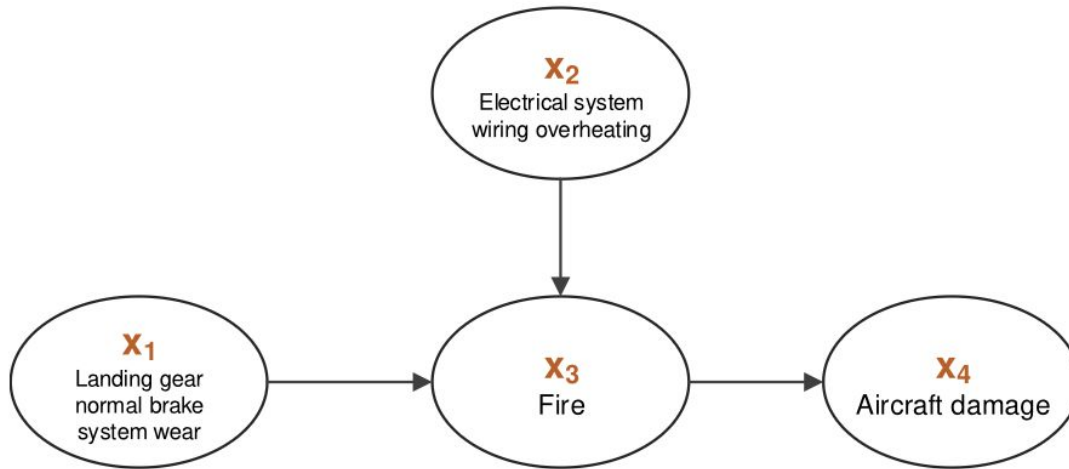
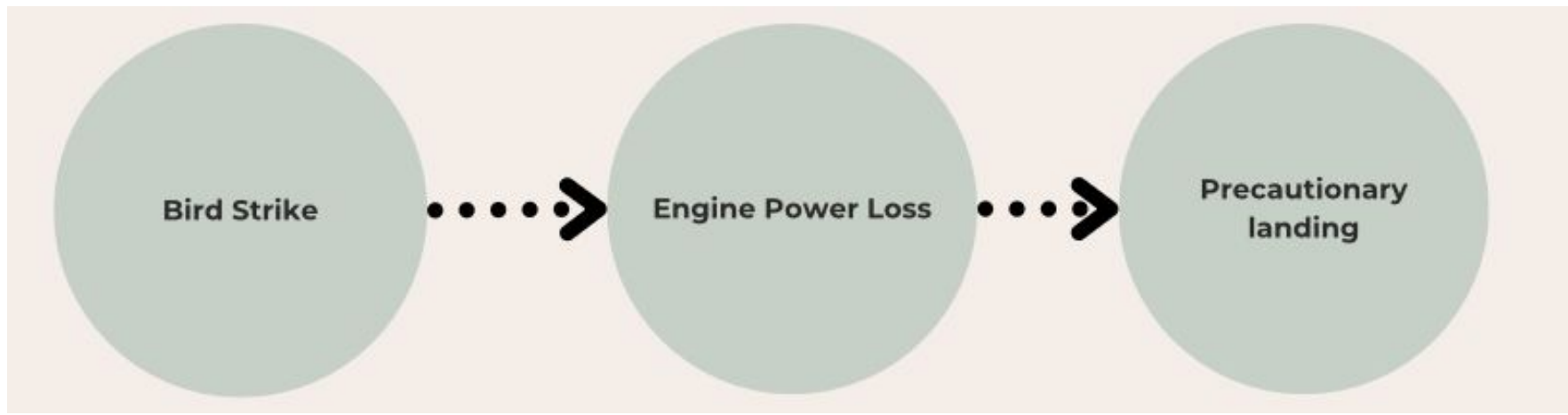


Fig. 2. A simple Bayesian network illustrating the occurrence of an aviation accident.



What is a Bayesian Network?

- Nodes = events/conditions
- Arrows = dependencies (cause → effect)
- Inference = forward or backward
- $P(\text{Engine Power Loss} \mid \text{Bird Strike}) = 0.40$



Extending Prior Work

- Same dataset & timeframe (1982–2016, FAR 121, accidents)
- Prior work: coded data + BN inference
- My prior work: pure LLM approach
- My extension: coded + narratives (structured + unstructured)
- LLM-based reasoning → backward & forward inference





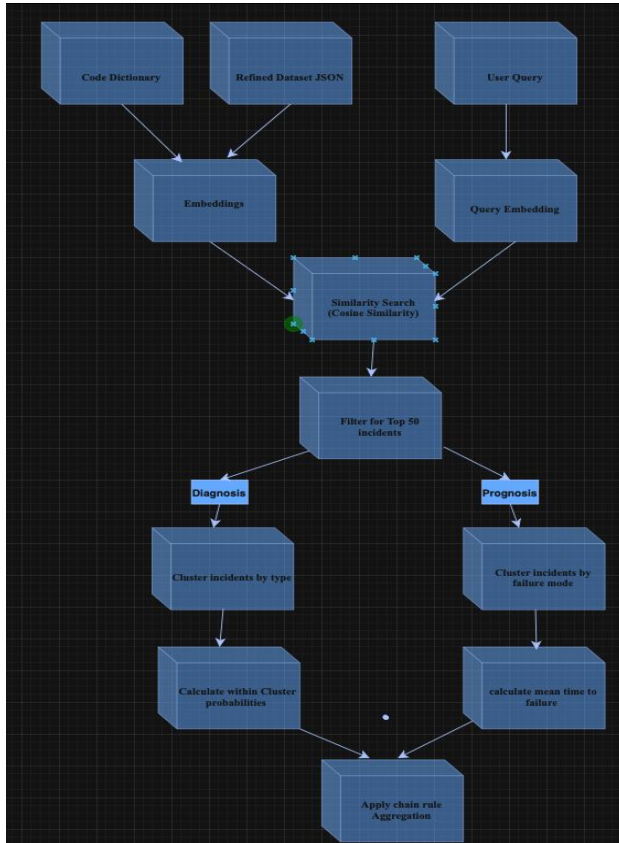
Previous approach & Limitations

- Previous approach:
 - Full LLM approach
- Limitations of the previous approach:
 - LLM inference is not the same as explicit Bayesian probabilities
 - Risk of hallucinations or overconfident outputs
 - Sensitive to input phrasing and prompts
 - Computational challenges at scale

Model Structure Overview



- **Diagnosis Mode:**
 - Analyzes completed incidents to understand what caused accidents
 - Identify prevention strategies for precursors
- **Prognosis Mode:**
 - Forecasts how current symptoms might escalate into future failures
 - Suggest preventive actions for downstream events



Merged Dataset JSON



- Data is scattered in 14 different files
 - Aircraft, sequence of events, engines, findings, injury, narratives, and occurrences file
- Solution: We merged all files using common EvID column name into a Merged Dataset JSON file
 - This will allow for faster retrieval in results when running the streamlit app
- Total: ~2,243 incident reports

Embeddings



- Tool: OpenAI's text-embedding-3-small
- What it does: converts any text $\rightarrow$ 1,536 dimensional numerical vector
- Each number captures a different meaning
- Similar text $\rightarrow$ similar vectors $\rightarrow$ close together in space
- "Engine failure during cruise" $\longleftrightarrow$ "Loss of power in flight" = High similarity
- "Engine failure during cruise" $\longleftrightarrow$ "Bird strike on runway" = Low similarity



Finding Similar Incidents

- Score every incident against the query: 0 (unrelated) $\rightarrow$ 1 (identical)
- Formula: $\text{similarity}(q,i) = (q \times i) / (|q| \times |i|)$
- Take top 50 most similar incidents
- Example query: engine lost power during flight
 - 0.87: engine failure in cruise
 - 0.82: total power loss
 - 0.25: bird strike on ground

Clustering



- Problem: top 50 incidents are a mix of different failure types
- Solution: group them by incident type automatically
- Setup: K-means groups all 2,243 incidents into 50 clusters, LLM names each cluster
- Query time: look up which cluster each of the top 50 belong to
- Ex:
 - 12 incidents → engine failure
 - 8 incidents → fuel system
 - 5 incidents → landing gear

Finding cause Frequencies in each Cluster



- Goal: $P(\text{Cause} \mid \text{Cluster}) \rightarrow$ given this incident type, how likely is each cause
- Note: One incident can have multiple causes
- Process: Extract all causes $\rightarrow$ Count frequency of each cause $\rightarrow$ Divide by total causes in cluster

Apply chain rule



- Goal: Calculate final $P(\text{Cause} \mid \text{Query})$ across all clusters
- Formula: $P(\text{cause} \mid \text{Query}) = \sum P(\text{Cause} \mid \text{Cluster}) \times P(\text{Cluster} \mid \text{Query})$
- $P(\text{Cluster} \mid \text{Query})$: Weight = average similarity x incident count, then normalize
- More similar + more incidents $\rightarrow$ more influence on Final probability

Prognosis:



- Step 1-4 are identical to diagnosis
 - Merged dataset → embeddings → similarity search → clustering
- Difference: instead of extracting causes, we look at sequence of events
- Prognosis asks: what happens next?

Sequence of Events Data



- Each NTSB incident has an ordered chain of events ($1 \rightarrow 2 \rightarrow 3 \dots$)
- One event is marked as the defining event, the anchor point
- Before anchor: what led to this event?
- After anchor: what does this event lead to?



Finding Event Frequencies in each cluster

- $P(\text{NextEvent} \mid \text{Cluster})$ = given this type of incident, what event happens next?
- Process:
 - 1. Take one cluster (ex: turbine blade failure with 12 incidents)
 - 2. For each incident, find the defining event
 - 3. Record what event comes immediately after
 - 4. Count frequency of each unique next event
 - 5. Calculate probability = count / total sequences

Apply chain rule



- Goal: calculate final $P(\text{NextEvent} \mid \text{query})$ across all clusters
- Formula : $P(\text{NextEvent} \mid \text{Query}) = \sum P(\text{NextEvent} \mid \text{Cluster}) \times P(\text{Cluster} \mid \text{Query})$
- Same weighting as diagnosis mode: Weight = average similarity x incident count, then normalize
- Only difference from diagnosis: next events replace causes



Multi step Event sequence prediction

- Step 1 gives immediate next event, but events cascade: $A \rightarrow B \rightarrow C \rightarrow D$
- Steps 2 & 3 use transition probabilities between consecutive events
- $P(\text{Event at step } t \mid \text{Query}) = \sum P(\text{Event at step } t \mid \text{Event at step } t-1) \times P(\text{Event at step } t-1 \mid \text{Query})$
- Built from historical data: $P(\text{Fire/smoke} \mid \text{Engine shutdown}) = 7/20 = 35\%$
- Same concept as forward inference in a bayesian network

Live Demo



Comparison with Zhang & Mahadevan (2021)



	Zhang & Mahadevan (2021)	My pipeline
EX 1: Pilot Error	Pilot error → substantial damage: 24.64%	Top cause: Pilot error: 21.7%
EX 2: Engine power	Engine instruments → forced landing: 13.57%, serious injury: 6.23%	Powerplant malfunction → fire/smoke → cabin safety event: 6%

- Same escalation story, different starting point
- Zhang propagates probabilities through fixed coded notes; my pipeline predicts the event sequence from natural language input
- Zhang needs coded node inputs; my system takes text
- Diagnosis mode is demonstrated here for the first time

Limitation

- Small clusters can inflate probabilities due to limited data
- New incidents require re running the full pipeline to incorporate it



Future work



- ModernBERT: improved fine tuning approach built and ready to test, the goal is to match OpenAI quality with a free local model
- Anthropic Skills: explore using structured AI skills/tools to improve query understanding and reasoning



Thank you!

Appendix





Example: Engine failure cluster

- Concrete example: Engine fair cluster
 - 12 incidents retrieved
 - Total cause extracted: 20 (some incidents have 1 cause, some have 3)
- Cause frequencies:
 - Improper maintenance → appears 7 times
 - Fuel contamination → appears 5 times
 - Pilot error → appears 2 times
 - Manufacturing defect → appears 3 times
- Calculations:
 - $P(\text{improper maintenance} \mid \text{engine failure}) = 7/20 = 0.35$ (35%)
 - $P(\text{fuel contamination} \mid \text{engine failure}) = 5/20 = 0.25$ (25%)
 - $P(\text{pilot error} \mid \text{engine failure}) = 3/20 = 0.15$ (15%)



Chain Rule - Worked Example

Complete example: calculating $P(\text{improper maintenance} \mid \text{Query})$

Setup: top 50 incidents grouped into 3 clusters

Engine failure: 12 incidents, avg similarity = 0.85

Fuel system: 8 incidents, avg similarity = 0.75

Landing gear: 5 incidents, avg similarity = 0.60

Step 1: Calculate $P(\text{Cluster} \mid \text{Query})$ for each cluster

Cluster Weights:

- Engine Failure: $0.85 \times 12 = 10.2$
- Fuel System: $0.75 \times 8 = 6.0$
- Landing Gear: $0.60 \times 5 = 3.0$
- Total Weight = $10.2 + 6.0 + 3.0 = 19.2$

continued



Step 2: Get $P(\text{"Improper Maintenance"} | \text{Cluster})$ from earlier calculation

- $P(\text{Improper Maintenance} | \text{Engine Failure}) = 0.35$ (35%)
- $P(\text{Improper Maintenance} | \text{Fuel System}) = 0.20$ (20%)
- $P(\text{Improper Maintenance} | \text{Landing Gear}) = 0.10$ (10%)

Step 3: Apply Chain Rule

$$\begin{aligned} P(\text{"Improper Maintenance"} | \text{Query}) &= \\ & (0.35 \times 0.53) + (0.20 \times 0.31) + (0.10 \times 0.16) \\ &= 0.186 + 0.062 + 0.016 \\ &= 0.264 = 26.4\% \end{aligned}$$

Final Answer: "Improper Maintenance" has 26.4% probability as root cause



Clustering

- Problem: raw similarity gives us a mix of incidents. We need to organize them
- Solution: used LLM GPT-4o-mini to sort top 50 incidents into distinct groups on the fly

Diagnosis:

- Group by Incident type
- Goal: create scenarios to check for root causes

Prognosis:

- Group incidents by failure mode
- Goal: create scenarios to check for component lifespan



Example - Turbine blade failure cluster

- 12 incidents in cluster, 10 had usable event sequences
- Total next events extracted: 10
- Next event frequencies:
 - Engine shutdown → appears 4 times
 - Powerplant malf/fail → appears 3 times
 - fire/smoke (non impact) → appears 2 times
 - Loss of engine power → appears 1 time
- Calculations:
 - $P(\text{Engine shutdown} \mid \text{Turbine blade failure}) = 4/10 = 0.40 = 40\%$
 - $P(\text{powerplant malf/fail} \mid \text{turbine blade failure}) = 3/10 = 0.30 = 30\%$
 - $P(\text{fire/smoke} \mid \text{Turbine blade failure}) = 2/10 = .20 = 20\%$
 - $P(\text{loss of engine power} \mid \text{turbine blade failure}) = 1/10 = .10 = 10\%$



Sequence of Events Example

- Incident 54642545 (from turbine blade failure cluster):
 - Event 1: enroute-cruise - powerplant sys/comp mal/ fail
 - Event 2: enroute-cruise - Engine shutdown ← defining event
 - Event 3: initial climb - fire/smoke (non impact)
 - Event 4: emergency descent - Loss of engine power (total)
 - Anchor = Event 2 (engine shutdown)
 - Downstream events = Event 3 and 4
 - Immediate next event = Event 3 (fire/snoke)
- Another incident 4968459 (also turbine blade failure cluster):
 - Event 1: initial climb - power plant sys/comp mal/ fail ← defining event
 - Event 2: initial climb - Engine shutdown
 - Event 3: Initial climb - Fire/smoke (non impact)
 - Anchor = Event 1
 - Immediate next event = Event 2 (engine shutdown)



Multi step example

Query: "loss of engine power during cruise flight"

Step 1 (Immediate next event):

- Engine shutdown: 33.9%
- Powerplant malf/fail: 16.8%
- Fire/smoke: 11.2%

↓ chain forward using transition probabilities

Step 2 (What follows Step 1):

- Fire/smoke (non-impact): 14.9%
- Loss of engine power: 4.2%
- Sys/Comp malf/fail: 3.2%

↓

Step 3 (What follows Step 2):

- Cabin safety event: 6.5%
- Powerplant malf/fail: 2.6%
- Engine shutdown: 1.9%

Full chain: Power loss → Shutdown → Fire/smoke → Cabin safety event

Modern BERT





ModernBERT! Why we explored it

- Currently using: OpenAi's text embedding 3 small
 - Requires internet and API key
 - Costs money per API call
 - 1,536 dimensions per embedding
 - Max 8,192 tokens per input
- Jesse Recommendation: try ModernBERT
 - Open source (runs locally, free, no API needed)
 - 768 dimensions per embedding
 - Max 8,192 tokens (same as OPENAI)
 - Original BERT only supports 512 tokens, ModernBERT is a major upgrade
- GOAL: can we match openAI's quality with a free local model?
-

What is fine tuning



- A pretrained model like ModernBERT is trained on general text
 - It understands English but not aviation specific language
 - It doesn't know that engine failure and loss of power are related
- Fine tuning = additional training on our specific data
 - We take pre trained model and train it further on NTSB data
 - This teaches it aviation specific relationships
- What changes:
 - The models internal weights are adjusted
 - Similar aviation concepts move closer in vector space
 - Different concepts move further apart
 - Engine failure $\longleftrightarrow$ loss of power = closer
 - Engine failure $\longleftrightarrow$ bird strike = further



How we fine tuned

- FIRST ATTEMPT (Tested — results on next slides):
 - Training pairs: 1,526 (same cluster = similar, different cluster = dissimilar)
 - Loss: CosineSimilarityLoss
 - Epochs: 3
 - Result: improved but not enough → designed better approach ↓
- IMPROVED APPROACH (Built, Ready to Test):
 - Training Data — 4 Types of Pairs:
 - 1. Cluster Label → Narrative
 - 2. Cause → Narrative
 - 3. Synthetic Query → Narrative
 - 4. Same-Cluster Narrative Pairs
 - Total: ~4,160 training pairs
 - Training Setup:
 - Loss: MultipleNegativesRankingLoss (MNRL)
 - Each batch of 32 gives 31 "free" negatives
 - Epochs: 3 | Batch size: 32 | Max tokens: 512
 - Status: ready to run, not yet tested



Fine tuning sanity check

After fine-tuning, we tested with sample pairs:

Should be HIGH similarity (related):

- "engine failure during cruise" $\leftrightarrow$ "loss of power in flight" $\rightarrow$ 0.733 ✓
- "fuel contamination in tank" $\leftrightarrow$ "contaminated fuel supply" $\rightarrow$ 0.785 ✓
- "bird strike on takeoff" $\leftrightarrow$ "collision with birds during departure" $\rightarrow$ high ✓
- "landing gear collapse" $\leftrightarrow$ "gear failure during landing" $\rightarrow$ high ✓

Should be LOW similarity (unrelated):

- "engine failure" $\leftrightarrow$ "bird strike on runway" $\rightarrow$ 0.626
 - "fuel contamination" $\leftrightarrow$ "landing gear collapse" $\rightarrow$ 0.438 ✓
 - "turbulence during cruise" $\leftrightarrow$ "hydraulic system leak" $\rightarrow$ low ✓
-
- Model learned aviation relationships
 - But: 0.626 for unrelated pairs is still too high
(ideally should be below 0.3)



Chain rule worked example

- Complete example: calculating $P(\text{Engine Shutdown} \mid \text{Query})$
- Setup: top 50 incidents grouped into clusters (showing top 3)
 - Turbine Blade Failure: 12 incidents, avg similarity = 0.82
 - Engine Fire: 5 incidents, avg similarity = 0.79
 - Ice Ingestion: 5 incidents, avg similarity = 0.75
- Step 1: Calculate $P(\text{Cluster} \mid \text{Query})$ for each cluster
- Cluster Weights:
 - Turbine Blade Failure: $0.82 \times 12 = 9.84$
 - Engine Fire: $0.79 \times 5 = 3.95$
 - Ice Ingestion: $0.75 \times 5 = 3.75$
 - Total Weight = $9.84 + 3.95 + 3.75 = 17.54$
 - $P(\text{Turbine Blade Failure} \mid \text{Query}) = 9.84 / 17.54 = 0.56$ (56%)
 - $P(\text{Engine Fire} \mid \text{Query}) = 3.95 / 17.54 = 0.23$ (23%)
 - $P(\text{Ice Ingestion} \mid \text{Query}) = 3.75 / 17.54 = 0.21$ (21%)



Chain rule - worked example continued

Step 2: Get $P(\text{"Engine Shutdown"} \mid \text{Cluster})$ from earlier calculation

- $P(\text{Engine Shutdown} \mid \text{Turbine Blade Failure}) = 0.40$ (40%)
- $P(\text{Engine Shutdown} \mid \text{Engine Fire}) = 0.50$ (50%)
- $P(\text{Engine Shutdown} \mid \text{Ice Ingestion}) = 0.00$ (0%)

Step 3: Apply Chain Rule

- $P(\text{"Engine Shutdown"} \mid \text{Query}) =$
- $= (0.40 \times 0.56) + (0.50 \times 0.23) + (0.00 \times 0.21)$
 - $= 0.224 + 0.115 + 0.000$
 - $= 0.339 = 33.9\%$
- Final Answer: "Engine Shutdown" has 33.9% probability as the next event after "loss of engine power during cruise flight"
- Repeat for every possible next event to get full ranked list



Multi step worked example calculating step 2

Building transition probabilities from sequence data:

- Look at every sequence where "Engine shutdown" appears
- Count what comes after it

Transition probabilities for top Step 1 events:

- $P(\text{Fire/smoke} \mid \text{Engine shutdown}) = 0.35$
- $P(\text{Fire/smoke} \mid \text{Powerplant malf/fail}) = 0.15$
- $P(\text{Fire/smoke} \mid \text{Fire/smoke}) = 0.05$

Calculating $P(\text{"Fire/smoke" at Step 2} \mid \text{Query})$:

$$\begin{aligned} & - P(\text{"Fire/smoke" at Step 2}) = \\ & - P(\text{Fire/smoke} \mid \text{Engine shutdown}) \times P(\text{Engine shutdown} \mid \text{Query}) \\ & + P(\text{Fire/smoke} \mid \text{Powerplant malf/fail}) \times P(\text{Powerplant malf/fail} \mid \text{Query}) \\ & + P(\text{Fire/smoke} \mid \text{Fire/smoke}) \times P(\text{Fire/smoke} \mid \text{Query}) \\ & = (0.35 \times 0.339) + (0.15 \times 0.168) + (0.05 \times 0.112) \\ & = 0.119 + 0.025 + 0.006 \\ & = 0.149 = 14.9\% \end{aligned}$$

Comparison results



Query: loss of engine power during cruise flight

	OpenAI	Base ModernBERT	Fine tuned ModernBERT
Top cluster	Turbine Blade Failure 43.5%	Clear Air Turbulence 24%	Taxiway collision 28%
Relevant 2nd cluster	Correct engine fire 12.5%	Wrong, turbulence related	Still wrong, pilot error 20%
Max similarity	~80-93%	N/A	~80-82%

openAI: correctly identifies engine related failure modes

Base ModernBERT: completely irrelevant

Fine tuned: improved but needs to be fine tuned more

Data



- ~25,000 accident records
- Detailed narratives: pilot/crew descriptions of what happened
- Event file: step by step chains showing how incidents escalated to final outcomes
- Code dictionary: translates numeric codes into aviation meaning
 - (e.g. 1452 → pilot error)
- Aircraft & engine file: performance history, maintenance records, and operational details
- Findings file: early warning signs and precursor conditions that led to incidents
- Occurrences file: timing and progression details for each accident phase



